BANKING B · BREAK THROUGH TECH × MASTERCARD

Feature Engineering

Turning the EDA into model-ready data for 1.1 million online card transactions

Read the full write-up

What we hand to the modeling pipeline

537
columns for tree models

515
normalized columns for a neural net

6
teammate notebooks combined

Where we are: feature engineering combined into one notebook

01

Six notebooks

Eric, Jean, Jennifer, Salvador, Susie and Sal each engineered features on their own.

02

Built on the EDA

Same raw data, same join, same 472,432 training transactions.

03

Combined

One final notebook keeps the best version of every feature and credits each author.

04

Checked and saved

Leakage checks pass; tree and neural-net tables are saved for modeling.

Result: the immediate blocker before model training is cleared.

We split by time and kept a labeled holdout for the final score

Period	Rows	Days	Fraud	Fraud rate	Share of fraud	Used for
Train	472,432	1 to 141	≈16,598	3.51%	80.3%	Learning rules
Validation	59,054	141 to 161	1,851	3.13%	9.0%	Comparing
Holdout	59,054	161 to 182	≈2,214	3.75%	10.7%	Final score
All labeled	590,540	1 to 182	20,663	3.50%	100%	
Kaggle test	506,691	213 to 395	no labels	no labels	no labels	Submission only

≈: from the printed rates; validation's count is exact. Boundaries fall between timestamps, so payments made in the same second never land in two periods.

Fraud rate

fraud ÷ rows

Validation: 1,851 ÷ 59,054 = 3.13%

Share of fraud

period fraud ÷ all labeled fraud

Validation: 1,851 ÷ 20,663 = 9.0%

Kaggle test has no labels, so it can't give detection rate or TFPR; the holdout is the labeled out-of-time set the advisor asked for.

Every piece of the advisor’s guidance has a home in the notebook

Debasmita’s guidance	How we handled it
Avoid one-hot for columns with many categories	Frequency encoding for the 14 many-category columns
Mean encoding works only if fraud rates stay stable	Jennifer’s out-of-time target encoding for stable columns
Trees need derived features; neural nets need normalization	Two outputs: a tree table and a normalized neural-net table
Start with all features, reduce within 1 to 2%	Named feature sets, including a reduced-V set
Judge models by detection rate and TFPR in a KS table	Reusable KS table, demoed on one feature
Keep a config per run	Config saved next to every output

Some columns were frequency-encoded

10 or Less Distinct Values → One-Hot Encoding

Includes columns such as ProductCD, card4, card6, DeviceType, M4.

More Than 10 Distinct Values → Frequency Encoding

Each values becomes its share of training transactions: gmail.com = 0.38. Never-seen values get 0. Uses no fraud labels.

Mean encoding: added for stable columns by Jennifer (slide 9). addr1 is left out: Sal's stability check found its fraud ranking reshuffled over time (0.38).

14 Columns

new columns for the 14 many-category columns: one-hot vs frequency encoding

Frequency also gives raw codes a meaningful order, from rare to common.

Amounts are judged against their own product's typical amount

The same 150 payment is **3.0×** typical on product H (150 ÷ 50) but only **1.2×** on product R (150 ÷ 125).

Feature	Calculation	Example	Kind	From
amount_to_product_median	amount ÷ max(product's training median, 1)	150 on H: 150 ÷ 50 = 3.0	Interaction	Eric · EDA
amount_train_frequency	training rows with this exact amount (to the cent) ÷ all training rows	never-seen amount = 0	Frequency	Eric · EDA
amount_fractional_unit	amount − whole-number part	25.375 → 0.375	Transform	Eric
amount_has_subcent_precision	1 if amount × 100 is not a whole number	25.375 → 1; 25.37 → 0	Transform	Eric · Salvador
amount_log1p	log(1 + amount)	500 → 6.22	Transform	Eric
hour_sin, hour_cos	hour = (seconds ÷ 3,600) mod 24; sin and cos of (2π × hour ÷ 24)	hour 6 → 1.00, 0.00	Transform	EDA
weekday_sin, weekday_cos	day = (seconds ÷ 86,400) mod 7; sin and cos of (2π × day ÷ 7)	day 0 → 0.00, 1.00	Transform	Eric · Salvador

Interaction or latent? Only amount_to_product_median is an interaction (amount combined with product). The rest transform or look up one column. None are latent variables; the closest are the missingness clusters on slide 11.

Account keys let us compare each payment to its account's past

Key 1: card1 | addr1 | D1n · the EDA's key, needs an address

Key 2: card1 | email | D1n · new, works without an address

	Time (s)	Amount	Device	Prior count	Usual amount	× usual	Secs since last	Devices before
T1	1,000	50	windows	0	missing	missing	missing	0
T2	5,000	40	windows	1	50	0.80	4,000	1
T3	9,000	200	samsung	2	45	4.44	4,000	1
T4	9,000	60	windows	2	45	1.33	4,000	1
T5	20,000	500	ios	4	87.50	5.71	11,000	2

Only earlier timestamps count, and fraud labels are never used. T3 and T4 share a timestamp, so neither sees the other. T5: almost 6× the usual amount, on a device the account never used.

Formulas, worked for T5

Prior count = earlier transactions
T1 to T4 = 4

Usual = earlier total ÷ prior count
(50 + 40 + 200 + 60) ÷ 4 = 87.50

× usual = amount ÷ usual
500 ÷ 87.50 = 5.71

Secs since last = time − last earlier time
20,000 − 9,000 = 11,000

Devices before = different earlier devices
windows, samsung = 2

Key 2 gives history to the group that holds 37% of fraud

Training group	Transactions	Fraud	Fraud rate	Share of fraud	Key 1 history	Key 2 history
Address missing	53,761	≈6,127	11.4%	36.9%	0%	58.3%
Address present	418,671	≈10,471	2.5%	63.1%	59.9%	49.3%
All training	472,432	≈16,598	3.51%	100%		

Address-missing transactions only, by Key 2 history:

Key 2 history?	Transactions	Fraud	Fraud rate
No	22,395	≈2,385	10.65%
Yes	31,366	≈3,742	11.93%
Total (address missing)	53,761	≈6,127	11.4%

≈: worked back from the printed rates. The yes/no flag is the bluntest feature; amount and device changes should matter more.

0% → 58%

of address-missing transactions now have account history

Fraud rate = fraud ÷ transactions
6,127 ÷ 53,761 = 11.4%

Share of fraud = group fraud ÷ all fraud
6,127 ÷ 16,598 = 36.9%

Key 2 history = with history ÷ group
31,366 ÷ 53,761 = 58.3%

Target encoding makes card2 the strongest single feature

Jennifer's **target (mean) encoding** replaces each category with its smoothed training fraud rate, using only earlier time folds for training rows.

Encoded column	Stability	Validation AUC
te_card2	0.48	0.728
te_R_emaildomain	0.68	0.683
te_P_emaildomain	0.31	0.589

Stability: Spearman rank correlation

$1 - (6 \times \text{sum of } d \cdot d) \div (m \times (m \times m - 1))$

d = a category's rank in half 1 minus its rank in half 2; m = number of categories. Ranks 1, 2, 3, 4 become 1, 3, 2, 4: d = 0, −1, 1, 0, so sum of d·d = 2 and $1 - (6 \times 2) \div (4 \times 15) = 0.80$.

AUC: ranking quality

pairs where fraud scores higher ÷ all fraud–legit pairs

Validation: 1,851 × 57,203 ≈ 105.9M pairs. te_card2 ranks the fraud higher in 72.8%. Ties count half.

addr1 is left out because its ranking drifts (0.38 across four training periods).

Plus 32 account-behavior features on both keys

Repeat amount · 0.612

Same exact amount paid before

Velocity · 0.601

Last hour, day, week and month

Amount deviation · 0.585

Unusual for this account, new record

New value · 0.581

Device, email or address never used

How the fraud rates behind target encoding are calculated

One column's categories in the first half of training (illustrative numbers):

Category	Transactions	Fraud	Fraud rate	Share of fraud	Encoding	Second-half rate
A	500	45	9.0%	20.0%	8.8%	8.1%
B	2,000	84	4.2%	37.3%	4.2%	4.9%
C	3,000	60	2.0%	26.7%	2.0%	2.6%
D	4,500	36	0.8%	16.0%	0.8%	1.1%
Total	10,000	225	2.25%	100%		

Stability: A, B, C, D keep the same order in both halves, so the rank correlation is 1.0. Real results: R_emaildomain 0.68, card2 0.48, P_emaildomain 0.31.

Fraud rate

fraud ÷ transactions

A: 45 ÷ 500 = 9.0%

Share of fraud

category fraud ÷ all fraud

A: 45 ÷ 225 = 20.0%

Encoding: the general smoothed rate

$$\text{smoothed} = w \times \text{observed} + (1 - w) \times \text{prior}$$

$$w = n \div (n + m)$$

n = transactions, m = 20, prior = 3.5%. Same as (fraud + m × prior) ÷ (n + m). A: w = 500 ÷ 520 = 0.96, so 0.96 × 9.0% + 0.04 × 3.5% = 8.8%.

Same idea elsewhere: credibility weighting in insurance pricing (Z = n ÷ (n + k)), the Beta-Binomial posterior mean in Bayesian statistics, and Laplace smoothing in machine learning. Trust the data more as it grows; lean on the average when it's thin.

Missing data is a signal, so we describe it three ways

Which: 24 block flags

How much: 4 missing counts

Pattern: 3 k-means clusters

Cluster	Transactions	Share of data	Fraud	Fraud rate	Share of fraud	EDA fraud rate
0	354,147	75.0%	≈7,543	2.13%	45.5%	2.1%
1	68,414	14.5%	≈3,038	4.44%	18.3%	4.4%
2	49,871	10.6%	≈6,010	12.05%	36.2%	12.0%
Total	472,432	100%	≈16,591	3.51%	100%	

Cluster 2 is 10.6% of training but 36% of its fraud. ≈: fraud counts worked back from the printed rates, so they add to about 16,591 rather than 16,598. Jean's clusters, fit on training only, match the EDA almost exactly.

Formulas, worked for cluster 2

Share of data = cluster ÷ all training

49,871 ÷ 472,432 = 10.6%

Fraud rate = fraud ÷ transactions

6,010 ÷ 49,871 = 12.05%

Share of fraud = cluster fraud ÷ all fraud

6,010 ÷ 16,591 = 36.2%

Two output tables, because trees and neural nets need different inputs

	Tree table (537 columns)	Neural-net table (515 columns)
Numbers	Kept as they are	log for skewed columns, clipped, standardized
Missing values	Left missing	Filled with the median, plus missing flags
Few categories	Categories	One-hot
Many categories	Raw column plus frequency	Frequency only
V columns	All 339	210 after removing near-copies

raw · 430 · original

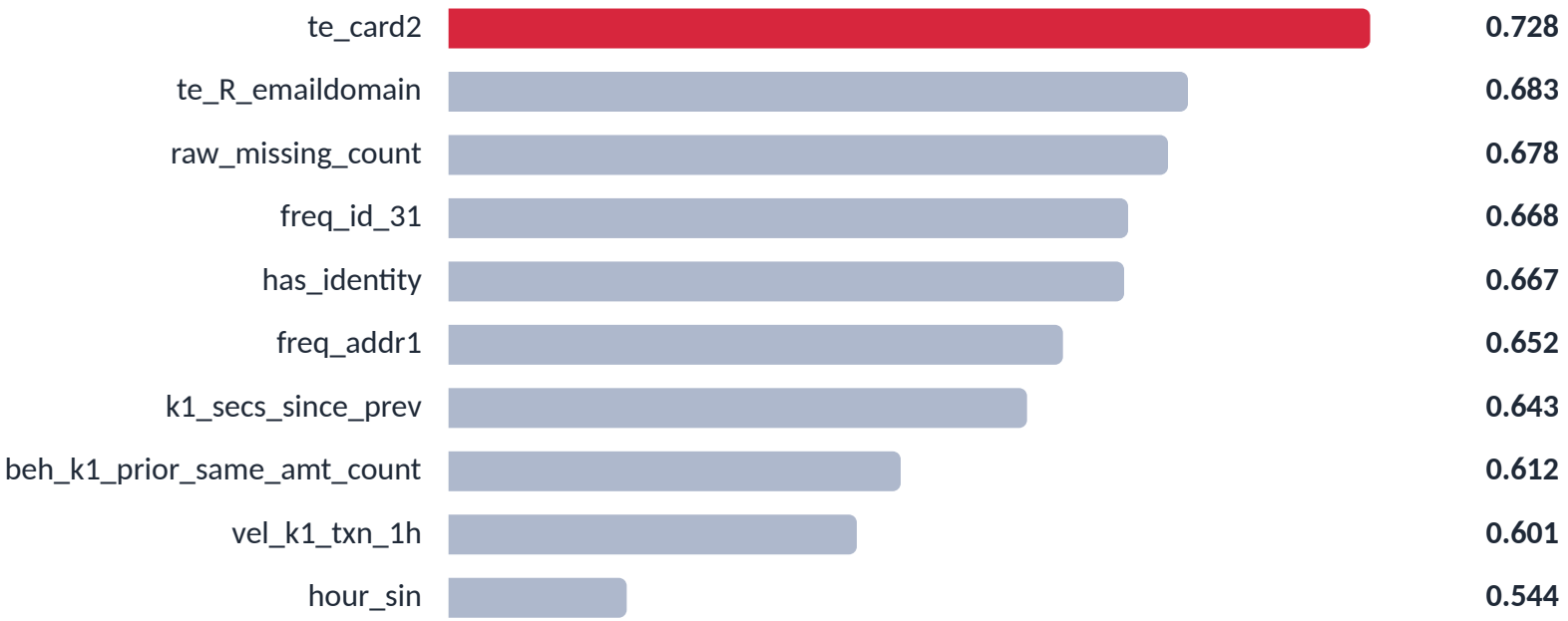
all_no_history · 489 · clean

all · 537 · with history

all_reduced_v · 408 · 1–2% test

On their own, target encoding and identity features carry the most signal

Validation ROC-AUC with each feature alone as the score. Bars start at 0.5, a coin flip.



Encoded card2 leads. Most of the rest repeat one signal: **whether a transaction has identity data**.

Low scores aren't useless: AUC misses bumps like the fraud peak in hours 5 to 10, and features combine.

Encoded card2 alone catches half the fraud at 11.3 flags per fraud

KS table for te_card2 on validation (1,851 fraud, 57,203 legitimate), the advisor's format:

Flag the top	Fraud caught	Detection rate	Legit flagged	Share of legit	TFPR	KS
10%	742	40.1%	5,164	9.0%	7.0:1	0.311
20%	958	51.8%	10,853	19.0%	11.3:1	0.328
30%	1,112	60.1%	16,604	29.0%	14.9:1	0.310
All (100%)	1,851	100%	57,203	100%	30.9:1	0

Sharper than raw_missing_count at the top 10% (35% of fraud at 8.1:1), about equal by 20%.

Target: about 10:1 TFPR at 50 to 60% detection. TFPR is legitimate transactions flagged per fraud caught, a fraud-industry ratio. The notes say "10%"; to confirm with Debasmita.

Formulas, worked for the top 20%

Detection = fraud caught ÷ all fraud
958 ÷ 1,851 = 51.8%

Share of legit = legit flagged ÷ all legit
10,853 ÷ 57,203 = 19.0%

TFPR = legit flagged ÷ fraud caught
10,853 ÷ 958 = 11.3:1

KS = detection − share of legit
0.518 − 0.190 = 0.328

The final notebook combines all six teammates' work

Teammate	In the final notebook
Eric	Time split, email and device groups, frequency encoding, amount and weekday features, Key 1 history, missingness counts, leakage checks, saved outputs
Jean	Frequency Encoding
Jennifer	Target (mean) encoding with a stability check, new device, email and address flags, velocity, account amount statistics, repeated amounts
Salvador	id_14 timezone as a category, three-decimal amount flag, Kaggle-discussion ideas
Susie	7-day and 30-day velocity windows; her other features match ones already in
Sal	Key 2, distinct-value history, block flags, reduced-V set, neural-net table, signal check, KS table, combining the notebooks

Every section of the notebook names whose work it builds on.

What we'll do next

Build the modeling pipeline	One owner, end to end; it loads the saved tables directly
Compare the feature sets	Raw, clean, history-enriched and reduced-V on validation
Report the advisor's metrics	AUC, PR-AUC and a KS table with detection rate and TFPR
Test the open choices	With and without Jennifer's block and Key 2; card codes as numbers or categories
Confirm with Debasmita	The 10:1 TFPR target and how much fraud teams flag
Score the holdout once	The final, honest number for the chosen model